

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 12

Controlling Program Flow

CHAPTER OVERVIEW

The if statement is probably the most used decision construct in Java. It allows you to execute a single statement or a block of statements if a particular condition is true.

EXAM OBJECTIVES COVERED

- ♦ if/else
- ♦ switch statements and expressions

Chapter 12 Controlling Program Flow

Exam Objectives

Create program flow control constructs including -

- if/else
- switch statements and expressions

12.1 Create if and if/else constructs

12.1.1 Basic syntax of if and if-else

The **if** statement is probably the most used decision construct in Java. It allows you to execute a single statement or a block of statements if a particular condition is true. If that condition is false, the statement (or the block of statements) is not executed. The following are two ways you can write an **if** statement:

```
if( booleanExpression ) single_statement;
```

Notice that there are no curly braces for the statement. The **if** statement ends with the semi-colon. If you have multiple statements that you want to execute instead of just one, you can put all of them within a block like this:

```
if ( booleanExpression ) {  
    zero or more statements;  
}
```

Observe that there is no semi-colon after the closing curly brace. It is not an error if

present though.

If-else statement 🙋

The **if-else** statement is similar to an **if** statement except that it also has an **else** part where you can write a statement that you want to execute if the **if** condition evaluates to false:

```
if( booleanExpression ) single statement; //semi-colon required here  
else single statement; //semi-colon required here, if-else statement  
ends with this semi-colon.
```

Again, if you have multiple statements to execute instead of just one, you can put them within a block:

```
if ( booleanExpression ) single statement; //semi-colon is required here  
else {  
    zero or more statements;  
}//no semi-colon required here, but not an error if it exists.
```

or, if both the **if** and the **else** parts have multiple statements:

```
if ( booleanExpression ) {  
    zero or more statements;  
} //must NOT have a semi-colon here, error if it exists.  
else {  
    zero or more statements;  
}//no semi-colon required here, but not an error if it exists.
```

If `booleanExpression` evaluates to `true`, the statement (or the block of statements) associated with **if** will be executed and if the `booleanExpression` evaluates to `false`, the **else** part will be executed.

Remember that an empty statement (i.e. just a semicolon) is a valid statement and therefore, the following if and if-else constructs are valid:

```
boolean flag = true;

if(flag); //does nothing, but valid

if(flag); else; //does nothing, but valid

if(flag);else System.out.println(true); //does nothing because flag is true

if(flag) System.out.println(true); else; //prints true
```

By the way, you may see **if/if-else** statements being called **if-then/if-then-else** statement. This is not entirely correct. In some languages such Pascal, "then" is a part of the syntax but it is not in Java. However, "then" is not a keyword in Java and there is no "then" involved in the syntax of **if/if-else**.

12.1.2 Usage of if and if-else in the exam 🙋

Let us now look at a few interesting ways if/if-else is used in the exam that might trip you up.

Bad syntax 🙋

```
boolean flag = true;
if( flag )
else System.out.println("false"); //compilation error
```

In the above code, there is no statement or a block of statements for the if part. If you don't want to have any code to be executed if the condition is true but want to have code for the else part, you need to put an empty code block for the if part like this:

```
boolean flag = false;
if( flag ) {
}
else System.out.println("false");
```

or

```
boolean flag = false;
if( flag ) ; else System.out.println("false");
```

Instead of having an empty if block, it is better to negate the if condition and put the code in the if block. Like this:

```
boolean flag = false;
if( !flag ) {
    System.out.println("false");
}
```

Bad Indentation

Unlike some languages such as Python, indentation (and extra white spaces, for that matter) holds no special meaning in Java. Indentation is used solely to improve readability of the code. Consider the following two code snippets:

```
boolean flag = false;
if(flag)
    System.out.println("false");
else System.out.println("true");
{
    System.out.println("false");
}
```

and

```
boolean flag = false;
if(flag)
    System.out.println("false");
else
    System.out.println("true");

{
    System.out.println("false");
}
```

The above two code snippets are equivalent. However, since the second snippet is properly indented, it is easy to understand that the last code block is not really associated with the if/else statement. It is an independent block of code and will be executed irrespective of the value of `flag`. This code will, therefore, print `true` and `false`.

Here is another example of bad indentation:

```
boolean flag = false;
int i = 0;
if(flag)
    i = i + 1;
    System.out.println("true");
```

else

```
i = i + 2;  
System.out.println("false");
```

The above code is trying to confuse you into thinking that there are two statements in the if part and two statements in the else part. But, with proper indentation, it is clear what this code is really up to:

```
boolean flag = false;  
int i = 0;  
  
if(flag) i = i + 1;  
  
System.out.println("true");  
  
else i = i + 2;  
  
System.out.println("false");
```

It turns out that the last three lines of code are independent statements. The else statement is completely out of place because it is not associated with the if statement at all and will, therefore, cause compilation error.

Missing else 🙄

As we saw earlier, the else part is not mandatory in an if statement. You can have just the if statement. But when coupled with bad indentation, an if statement may become hard to understand as shown in the following code:

```
boolean flag = true;  
if(flag)  
System.out.println("true");  
{
```

```
System.out.println("false");  
}
```

The above code prints `true` and `false` because there is no **else** part in this code. The second `println` statement is in an independent block and is not a part of the **if** statement.

Dangling else

"Dangling else" is a well known problem in programming languages that have **if** as well as **if-else** statements. This is illustrated in the following piece of code that has two **if** parts but only one **else** part:

```
int value = 3;  
if(value == 0)  
if(value == 1) System.out.println("b");  
else System.out.println("c");
```

The question here is with which **if** should the **else** be associated? There are two equally reasonable answers to this question as shown below:

```
int value = 3;  
if(value == 0) {  
    if(value == 1) System.out.println("b");  
}  
else System.out.println("c");
```

and

```
int value = 3;  
if(value == 0) {  
    if(value == 1)
```



```
        System.out.println("b");
    else
        System.out.println("c");
}
```

In the first interpretation, **else** is associated with the first **if**, while in the second interpretation, **else** is associated with the second **if**. If we go by the first interpretation, the code will print `c`, and if we go by the second interpretation, the code will not print anything. For a compiler, both are legally valid ways to interpret the code, which makes the code ambiguous.

Since neither of the interpretations is more correct than the other, Java language designers broke the tie by deciding to go with the second interpretation, i.e., the **else** is to be associated with the nearest **if**. That is why the above code does not print anything as there is no **else** part associated with the first **if**. Based on the above discussion, you should now be able to tell the output of the following code:

```
int value = 3;
if(value == 0)
if(value == 1)
    System.out.println("b");
else
    System.out.println("c");
else
    System.out.println("d");
```

Just follow the rule that an else has to be associated with the nearest if. The following is how the above statement will be grouped:

```
int value = 3;

if(value == 0){
    if(value == 1)
        System.out.println("b");
}
```

```
        else
            System.out.println("c");
    }
    else System.out.println("d");
```

It will print `d`.

Using assignment statement in the if condition 🙄

Recall that every assignment statement itself is a valid expression with a value of its own. Its type and value are the same as the ones of the target variable. This fact can be used to write a very tricky if statement as shown below:

```
boolean flag = false;
if(flag = true){
    System.out.println("true");
}
else {
    System.out.println("false");
}
```

Observe that `flag` is not being compared with `true` here. It is being assigned the value `true`. Thus, the value of the expression `flag = true` is true and that is why the **if** part of the statement is executed instead of the **else** part. While this type of code is not appreciated in a professional environment, a similar construct is quite common:

```
String data = null;
if( (data = readData()) != null ) //assuming that readData() returns a String
{
    //do something
}
```

The above code is fine because the assignment operation is clearly separated from the comparison operation. The value of the expression `data = readData()` is being compared with `null`. Remember that the value of this expression is the same as the value that is being assigned to `data`. Thus, the `if` body will be entered only if `data` is assigned a non-null value.

Using pre and post increment operations in the if condition 🙌

You will see conditions that use pre and post increment (and decrement) operators in the exam. Something like this:

```
int x = 0;
if(x++ > 0){ //line 2
    x--; //line 3
}

if (++x == 2){ //line 6
    x++; //line 7
}
System.out.println(x);
```

You can spot the trick easily if you have understood the difference between the value of an expression and the value of a variable used in that expression (I explained this in the "Using Operators" chapter). At line 2, `x` will be incremented to `1` but value of the expression `x++` is `0` and therefore, the condition will be evaluated to `false`. Thus, line 3 will not be executed. At line 6, `x` is incremented to `2` and the value of the expression `++x` is also `2`. Therefore, the condition will be evaluated to `true` and line 7 will be executed, thereby increasing the value of `x` to `3`. Thus, the above code will print `3`.

Remember that conditions are used in ternary expressions and loops as well, so, you need to watch out for this trick there also.

12.2 Create ternary constructs

12.2.1 The ternary conditional operator ?:

The ternary operator is not mentioned in the exam objectives explicitly (although it was mentioned explicitly in OCA 8 objectives). We haven't seen any one getting a question on it either. However, it is one of the commonly used operators in Java and from that perspective, it falls within the scope of the exam objectives. You should go through the following discussion only if you have time.

The syntax of the ternary operator is as follows:

```
operand 1 ? operand 2 : operand 3;
```

Operand 1 must be an expression that returns a `boolean`. The `boolean` value of this expression is used to decide which one of the rest of the two other operands should be evaluated. In other words, which of the operands 2 and 3 will be evaluated is conditioned upon the return value of operand 1. If the boolean expression given in operand 1 returns true, the ternary operator evaluates and returns the value of operand 2 and if it is false, it evaluates and returns the value of operand 3. From this perspective, it is also a **"conditional operator"** (as opposed to `&`, `|`, `!`, and `^`, which are really just **"logical"** operators).

Examples:

```
boolean sweet = false;
int calories = sweet ? 200 : 100; //assigns 100 to calories
boolean sweetflag = (calories == 100 ? false : true); //assigns false to
sweetflag

boolean hardcoded = false;
```

```
//assuming getRateFromDB() method returns a double.
double rate = hardcoded ? 10.0 : getRateFromDB(); //invokes method
getRateFromDB

String value = sweetflag ? "Sweetened" : "Unsweetened";

Object obj = sweetflag ? "Sweetened" : new Object();
```

The ternary conditional operator is often thought of a short form for the **if/else statement** but it is similar to the if/else statement only up to conditional evaluation of its other two operands part. Their fundamental difference lies in the fact that the ternary expression is just an **expression** while an if/else statement is a **statement**. As discussed earlier, every expression must have a value and so, must the ternary expression. Since the value of a ternary expression is the value returned by the second or the third operand, the second and third operands of the ternary operator can comprise any expression. As the example given above shows, they can also include **non-void method invocations**. There is no such restriction with the if/else statement. The following example highlights this difference:

```
boolean flag = true;
if(flag) System.out.println("true");
else System.out.println("false");
```

The above if/else statement compiles fine and prints `true` but a similar code with ternary expression does not compile.

```
flag ? System.out.println("true") : System.out.println("false");
```

The reason for non-compilation of the above code is two fold. The first is that a ternary expression is not a valid statement on its own, which means, you cannot just have a free standing ternary expression. It can only be a part of a valid statement such as an assignment. For example,

```
int x = flag ? System.out.println("true")
```

```
: System.out.println("false");
```

This brings us to the second reason. The second and third operands in this example are invocations methods that return `void`. Obviously, you cannot assign void to an int variable. In fact, you cannot assign void to any kind of variable. Therefore, it fails to compile.

Type of a ternary conditional expression 🙌

Now that we have established that a ternary conditional expression must return a value, all that is left to discuss is the type of the value that it can return. It can return values of three types: **boolean**, **numeric**, and **reference**. (There are no other types left for that matter!)

If the second and third operands are expressions of type boolean (or Boolean), then the return type of the ternary expression is boolean. For example:

```
int a = 1, b = 2;
boolean flag = a == b ? true : false; //ternary expression that
returns a boolean
```

If the second and third operands are expressions of numeric types (or their wrapper classes), then the return type of the ternary expression is the wider of the two numeric types. For example, `double d = a == b ? 5 : 10.0;`. Observe here that the second operand is of type `int` while the third is of type `double`. Since `double` is wider than `int`, the type of this ternary expression is `double`. You cannot, therefore, do `int d = a == b ? 5 : 10.0;` because you cannot assign a double value to an int variable without a cast.

If the second and third operands are neither of the above, then the return type of the ternary expression is reference. For example,

```
Object str = a == b ? "good bye" : "hello";
```

 Here operands 2 and 3 are

neither numeric nor boolean. Therefore, the return type of this ternary expression is a reference type.

The first two types are straight forward but the third type begs a little more discussion. Consider the following line of code:

```
Object obj = a == b? 5 : "hello";
```

Here, the second operand is of a numeric type while the third is of type `String`. Since this falls in the third category, the return type of the expression `a == b? 5 : "hello";` must be a reference. The question before the compiler now is what should be the type of the reference that is returned by the expression. One operand is an `int`, which can be boxed into an `Integer` object and another one is a `String` object. If the `boolean` condition evaluates to `true`, the expression will return an `Integer` object and if the condition evaluates to `false`, the expression will return a `String` object. Remember that the compiler cannot execute any code and so, it cannot determine what the expression will return at run time. Thus, it needs to pick a type that is compatible with both kind of values. The compiler solves this problem by deciding to pick the most specific common superclass (technically known as "least upper bound") of the two types as the type of the expression. In this case, that class is `java.lang.Object`. By selecting the most specific common super class, the compiler ensures that irrespective of the result of the condition, the value returned by this expression will always be of the same type, i.e., `java.lang.Object`, in this case. Here is an example, where the most specific common super class is not `Object`:

```
Double d = 10.0;
Byte by = 1;
Number n = a == b? d : by;
```

Here, the most specific common superclass of `Double` and `Byte` is `Number`. (Recall that all wrapper classes for integral types extend from `java.lang.Number`, which in turn extends from `java.lang.Object`). You can therefore, assign the value of the expression to a variable of type `Number`.

You should now be able to tell the result of the following two lines of code:

```
int value = a = b? 10 : "hello"; //1
```

```
System.out.println(a = b? 10 : "hello"); //2
```

As discussed above, the type of the expression `a = b? 10 : "hello";` is `Object`. Can you assign an `Object` to an `int` variable? No, right? Therefore, the first line will not compile. Can you pass an `Object` to the `println` method? Of course, you can. Therefore, the second line will compile and run fine.

The short circuiting property of?: 🙌

Depending on whether the value of operand one is true or false, either operand 2 or operand 3 is evaluated. In other words, if the condition is `true`, operand 3 is not evaluated and if the condition is `false`, operand 2 is not evaluated. In this respect, the ternary conditional operator is similar to the other two short circuiting operators, i.e., `&&` and `||`. However, there is an important difference between the two. While with `&&` and `||`, evaluation of both the operands is possible in certain situations, it is never the case with `?:` operator. `?:` evaluates exactly one of the two operands in all situations.

The short circuiting nature of `?:` provides a good opportunity for trick questions in the exam. For example, what will the following code print?

```
int x = 0;
int y = 1;
System.out.println(x>y? ++x : y++);
System.out.println(x+ " " +y);
```

This code prints:

```
1
0 2
```


Since the value of `x` is not greater than `y`, `x > y` evaluates to `false` and therefore, the ternary expression returns the value of the third operand, which is `y++`. Since `y++` uses post-increment operator, the return value of `y++` will be the current value of `y`, which is `1`. `y` will then be incremented to `2`. Observe that evaluation of the second operand is short circuited because the first operand evaluates to false. Therefore, `++x` is never executed. Thus, the second print statement prints `0 2`.

12.3 Use a switch statement

12.3.1 Creating a switch statement 🙌

A switch statement allows you to use the value of a variable to select which code block (or blocks) out of multiple code blocks should be executed. A switch statement can be written in two ways - the old way and the new way (since Java 14). Syntactically, the only difference between the two ways is that the old way uses a colon (`:`) to separate the case label from the case block, while the new way uses an arrow (`→`).

Writing a switch statement using colons 🙌

Here is a simple switch statement that uses the old syntax:

```
void main(String[] args){
    switch(args.length) { //switch block starts here
        case 0 : System.out.println("No argument");
                break;
        case 1 : {
                    System.out.println("Only one argument");
                    break;
                } //enclosing code within { } is optional

        //associating multiple case labels with the same block
        case 2, 3 : System.out.println("Two or three
```

```

arguments");
        break;

        //another way of associating multiple case labels with the
same block
        case 4: case 5 : System.out.println("Four or five arguments");
            break;

        default : System.out.println("Too many arguments!");
            break;

    } //switch block ends here

    System.out.println("All done.");
}

```

12.3.2 Components of a switch statement 🙋

There are five blocks of code in the above switch statement. The first four blocks of code are associated with a **case label** each and the last one is associated with the **default** label. Collectively, the `case` and the `default` labels are called "**switch labels**" and the code associated with each of the switch labels are called switch blocks. Depending on the value of the selector expression `args.length`, the control will enter the code block associated with that particular switch label and keep on executing statements until it finds a

`break` statement. For example, if the value is `0`, the control will enter the first code block. It will print `No argument` and then encounter the `break` statement. The `break` statement causes the control to exit the switch statement and move on to the next statement after the switch block, which prints `"All done"`.

If the value of the expression doesn't match with any of the case labels, the control looks for a block labelled **default** and enters that block. If there is no default block either, the control does not enter the switch block at all. Since this "switching" is done based on the expression specified in the switch statement (which is just `args.length` in this example), this expression is aptly called the "**selector expression**".

Operationally, this seems quite similar to a cascaded **if/else** statement. Indeed, the above code can very well be written using an if/else statement as follows:

```
void main(String[] args){
    int i = args.length; //using i here just for convenience

    if(i == 0) {
        System.out.println("No argument");
    } else if (i == 1) {
        System.out.println("Only one argument");
    } else if (i == 2 || i == 3) {
        System.out.println("Two or three arguments");
    } else {
        System.out.println("Too many arguments!");
    }
    System.out.println("All done.");
}
```

Well, why do you need a switch statement then, you may ask. To begin with, as you can see, an if/else statement is a lot more verbose than a switch statement. The switch version is also a little easier to comprehend than the if/else version. But underneath this syntactic ease lies a complicated beast. This is evident when we look at the moving parts involved in a switch statement more closely.

The selector expression 🙌

The expression written within `switch(<expression>)` is called the selector expression. Its value determines which case block will be executed. Prior to Java 21, a the switch's selector expression was restricted to only a few types (namely, `byte`, `short`, `char`, `int` and their wrapper types, `String`, and enums) but with Java 21 the only restriction that remains is that a selector expression must not return `long`, `float`, `double`, or `boolean`. To be more explicit, as of Java 21, the following types are allowed:

1. a limited set of integral types (`byte`, `char`, `short`, `int`), and their wrapper classes. Observe that even though `long` is an integral type, it cannot be the type of a switch variable. `boolean`, `float`, and `double` are not integral types anyway and therefore, cannot be the type of a switch variable either.
2. **enum** type
3. `java.lang.String` - Since Java 7, you may use a `String` expression as a selector expression.
4. Prior to Java 21, it was not possible to have `null` as a case constant in a switch statement. If the selector expression returned `null`, a `NullPointerException` would be thrown. This restriction was lifted in Java 21, so you may now do the following:

```
String str = null;
switch(str){
    case null → System.out.println("str is null!");//valid in
Java 21
    default → System.out.println(str);
    //the above two lines can also be combined into one
line:
    //case null, default → System.out.println("null or
default");
}
```

But if you use `null` in a switch label you need to use it before the `default` label

4. (if the default label is present) and the switch statement must be **exhaustive**. I will explain what exhaustive means shortly while talking about switch expressions.

Note that, to maintain backward compatibility, a `null` value still does not match the `default` label. It matches only the `null` label. Without the `case null` block, a `switch` statement will still cause a `NullPointerException` to be thrown if the selector expression returns `null`.

5. **Any reference type** - With Java 21, the switch statement (and switch expression) has been enhanced significantly. This enhancement is commonly referred to as "**pattern matching for switch**". Among other things, this enhancement allows the selector expression to return any reference type (including **all** wrapper types). Since there are several additional rules that apply in this case, I will discuss it detail in a separate section.

Compare this to an **if/else** statement where branching is done based on the value of a **boolean expression**. This limits an if/else statement to at most two branches. Of course, as we saw earlier, you can cascade multiple if/else statements to achieve multiple branches.

The switch labels 🙌

Case labels must consist of one or more **compile time constants** that are assignable to the type of the selector expression. For example, if your selector expression is of type `byte`, you cannot have a case label with a value that is larger than a `byte`. Similarly, if your selector expression is of type `String`, the case labels must be constant string values as illustrated in the following code:

```
void main(String[] args){
    switch(args[0]){
        case "1", "2" : System.out.println("one or two"); //valid because "1"
and "2" are compile time constants
        case "1"+"2" : System.out.println("one plus two"); //valid because
"1"+"2" is a compile time constant
```

```
case args[1] : System.out.println("same args");//will not
```

compile because `args[1]` is not a compile time constant

```
    case "abc".toUpperCase() : System.out.println("ABC");//will not compile
    because "abc".toUpperCase() is not a compile time constant
  }
}
```

Observe that `"1"` and `"1"+"2"` are compile time constants because the value of these expressions is known at compile time, while `args[1]` and `"abc".toUpperCase()` are not compile time constants because their values can only be determined at run time when the code is executed.

The interesting thing about case labels is that they are **optional**. In other words, a switch statement doesn't necessarily have to have a case label or the default label. The following are, therefore, superfluous yet valid switch statements.

```
switch(i){
    default : System.out.println("This will always be printed");
}
switch(i){
    //empty block. useless but valid.
}
```

Another point worth repeating here is that although it is very common to use a single variable as the selector expression but you can use any expression inside the switch. And when you talk of an expression, all the baggage of numeric promotion, casting, and operator precedence that we saw previously, comes along with it. You need to consider all that while checking the validity of case labels. For example, while the following code fails to compile:

```
byte b = 10;
switch(b){ //type of the selector expression here is byte
    case 1000 : //1000 is too large to fit into a byte
```

```
        System.out.println("hello!");  
    }
```

the following code compiles fine:

```
byte b = 10;  
switch(b+1){ //type of the selector expression here is now int due  
    to numeric promotion  
    case 1000 : //1000 can fit into an int  
        System.out.println("hello!");  
    }
```

The default block 🙌

There can be at most one default block in a switch statement. The purpose of the default block is to specify a block of code that needs to be executed if the value of the switch expression does not match with any of the case labels. Just like the case labels, this block is also **optional**.

The order of case and default blocks 🙌

Java does not impose any particular order for the case statements and the default block (except in special situations involving null and pattern matching, which I will talk about later). Thus, although it is customary to have the default block at the end of a switch block, you can have it even at the beginning. Similarly, Java does not care about the ordering of the case labels.

However, "does not care" does not mean "not important"! Ordering of case and default blocks becomes very important in combination with the use of the `break` statement as we will see next.

The break statement 🙌

I mentioned in the beginning that the switch labels determine the entry point into a

switch statement and the break statement determines the exit. That is true but the interesting thing is that even the break statement is **optional**. A case block does not necessarily have to end with a break. Let me modify the program that I showed you in the beginning:

```
void main(String[] args){
    switch(args.length) { //switch block starts here

        case 0 : System.out.println("No argument");
                //break;

        case 1 : System.out.println("Only one argument");
                //break;

        case 2 : System.out.println("Two arguments");
                //break;

        default : System.out.println("Too many arguments!");
                //break;

    }//switch block ends here

    System.out.println("All done.");
}
```

I have commented out all the break statements. Now, if you run this program **without any argument**, you will see the following output:

```
No argument
Only one argument
Two arguments
Too many arguments!
All done.
```

The control entered at the block labelled **case 0** (because `args.length` is `0`), and executed all the statements of the switch block...even the statements associated with other case blocks that did not match the value of `args.length`. This is called **"fall through"** behavior of a switch statement. In absence of a break statement, the control falls through to the next case block and the next case block, and so on until it

reaches the end of the switch statement. This feature is used when you want to have one code block to execute for multiple values of the switch expression. Here is an example:

```
char ch = 0;
int noOfVowels = 0;
while( (ch = readCharFromStream()) > 0) {
    switch(ch) {

        case 'a' :
        case 'e' :
        case 'i' :
        case 'o' :
        case 'u' :
            noOfVowels++;

        default :    logCharToWhatever(ch);
    }
}
System.out.println("Number of vowels received "+noOfVowels);
```

The above code logs each character it receives but increments `noOfVowels` only if the character received is a lower case vowel.

You will see questions in the exam on this behavior of the switch statement. So pay close attention to where in the switch block does the control enter and where it exits. Always check for missing break statements that cause the control to fall through to the next case block.

The following is a typical code snippet you may get in an exam. Try running it with different arguments and observe the output in each case:

```
public class TestClass {
    public static void main(String[] args){
        int i = 0;
```

```

switch(args[0]) {
    default : i = i + 3;
    case "2" : i = i + 2;
    case "0" : break;
    case "1" : i = i + 1;
}
System.out.println("i is "+i);
}
}

```

The fall through behavior of a switch statement does not get used a lot in Java. In fact, it is discouraged because it reduces code readability, but it has been used in interesting ways to optimize code in other languages. If you have time, you might want to google "duff's device" to see one such usage. This is, of course, not required for the exam.

The switch blocks 🙌

The code blocks associated with the switch labels are called **switch blocks**. In a switch statement written using the **old syntax**, the rules for a switch block are quite flexible. It may be empty, it could contain zero or more "expression statements", it could contain zero or more code blocks, or it may throw an exception. As explained in the **Operators** chapter expression statements are special kind of expressions that can be used as stand alone statements (they are pre/post increment/decrement, assignment, method invocation, instance creation expressions). So, here are a few valid switch blocks:

```

int i = 0;
switch(i){
    case 0 : m1(); //single expression statement
    case 1 : m1(); m2(); //multiple expression statements
    case 2 : { m1(); i++; } //single code block
    case 3 : { m1(); } { --i; } //multiple code blocks
}

```

```
case 4: //empty
case 5: throw new RuntimeException("Invalid value for i");
}
```

12.3.3 Writing a switch statement using the new syntax 🙌

If you don't want the fall through behavior in a switch block, there is a new syntax available since Java 14. Here is the example shown earlier but rewritten using the new syntax:

```
void main(String[] args){

    switch(args.length) {
        case 0 → System.out.println("No argument");
                //break; //invalid
        case 1 → {
                System.out.println("Only one argument");
                break; //valid, but redundant
            } //enclosing code within { } is optional

        case 2, 3 → System.out.println("Two or three arguments");

        //cannot do this
        //case 4 → case 5 → System.out.println("Four or five
arguments");//invalid

        //cannot mix → and : in the same switch
        //case 6 : System.out.println("Six arguments");//invalid

        default → System.out.println("Too many arguments!");
    }
}
```

```
}  
    System.out.println("All done.");  
}
```

The above code uses `→` instead of `:`. It prints only `No argument` if run without any argument. Other case blocks are skipped even though there is no `break` statement at the end of the code block associated with `case 0`.

Here are a few points worth noting from the above code about the new syntax:

1. **uses arrow -> instead of colon :**
2. **single expression statement or block:** Every switch label is associated with either a single expression statement or single a code block (i.e. a set of statements enclosed within `{` and `}`). If you use a single statement, you cannot write a `break` statement after that because, well, then you are not writing a single statement!
3. **No break statements:** The new syntax ensures that the switch statement breaks automatically after the expression or the block executes. It is valid to write a `break` statement in the code block but it is redundant because the control will exit out of the switch statement after completing the code block anyway. In other words, the new syntax does not have fall through behavior. In fact, it is not possible to simulate the fall through behavior using the new syntax.
4. **Multiple case values in the same case label:** The new syntax has only one way to allow multiple values to be associated with a case block as shown in this line above: `case 2, 3 → System.out.println("Two or three arguments");`.
5. **Cannot mix syntaxes:** Cannot mix old and new syntax in the same switch statement. You can, of course, mix them in the same code base.

Which switch syntax to use - old or new? 🙋

As you have observed in the examples above, there are subtle differences in both the syntaxes, which are not easy to keep in

mind while coding. Using both the syntaxes may introduce subtle bugs at worst and reduces code readability at best. So, it is advisable to stick to one syntax in a project. You can do all the things with either of the syntaxes. They are equally powerful but the new syntax may be less verbose in certain situations. I personally use the old syntax because most of our existing code uses old syntax and it has not been ported to Java 17 yet.

12.3.4 Using switch with enums

Enums are used very often in a switch statement. As explained in detail earlier, an enum is pretty much like a class except that it offers only a fixed number of predefined named instances. You cannot create new instances of an enum. Here is an example of an enum:

```
public enum ButtonState{  
    ON, OFF; //these are implicitly public, static, and final  
}
```

Let us see how it can be used in a switch statement:

```
public class TestClass{  
    public static void main(String[] args){  
        ButtonState bs = ButtonState.ON; //get a ButtonState somehow  
        switch(bs){  
            case ON: System.out.println("Button is ON"); break;  
            case ButtonState.OFF: System.out.println("Button is OFF");  
break;  
            default : System.out.println("Invalid Button state");  
        }  
    }  
}
```

```
}  
}
```

There is nothing special going on in the above code. It is like any other switch statement. The only point worth noting is that prior to Java 21, the case statements could only use `ON` and `OFF` instead of `ButtonState.ON` and `ButtonState.OFF`. So, case `ButtonState.ON` would cause a compilation error. But since Java 21, using `<enum name>.<constant>` is also valid.

In the exam, you may see questions that use the `DayOfWeek` enum defined in the `java.time` package. This enum defines seven obvious values for each day, from `MONDAY` to `SUNDAY`. You do not need to know the details of this enum to answer the question. You will be able to answer it if you stick to the basics of a switch statement. For example, can you tell what will the following code print for different days of a week?

```
import java.time.*;  
public class TestClass{  
    public static void main(String[] args){  
        DayOfWeek dow =  
        LocalDate.now().getDayOfWeek();//java.time.LocalDate.now() returns  
        current date  
        switch(dow){  
            case MONDAY: TUESDAY: System.out.println("MON/TUE"); break;  
            case WEDNESDAY: FRIDAY: System.out.println("WED - FRI");  
            default : System.out.println("SAT/SUN");  
        }  
    }  
}
```

Let me tell you that it will compile fine. Let us see what happens for a `MONDAY`. It has

a matching case, so, it will print `MON/TUE` and then break out of the case because of the `break;` statement. Now, for `TUESDAY`, do you see a matching case? No. `TUESDAY:` is not a case statement. `TUESDAY` is just a label applied to the print statement. So, the control will not enter here. It will go to default and print `SAT/SUN`. You may work out the output for other days similarly.

Let me now rewrite the above switch statement using the new syntax:

```
switch(dow){
    case MONDAY, TUESDAY → System.out.println("MON/TUE");
    case WEDNESDAY, FRIDAY → System.out.println("WED - FRI");
    default → System.out.println("SAT/SUN");
}
```

Can you tell how the output of this switch statement differs from the previous one? **Hint:** There is no fall through behavior when you use the new syntax, but the new syntax allows multiple case expressions in the same case.

12.4 Use a switch expression

12.4.1 Creating switch expressions 🙋

A switch statement is a bit restricted in the sense that it does not have a return value and thus, cannot be used within an expression. For example, you cannot do something like this with it:

```
//will not compile
double computeTax1(double income, int taxBracket){
    return income * switch(taxBracket){
        case 0 : 0.1; break;
        case 1 : 0.2; break;
        default : 0.3; break;
    };
}
```

```
}
```

In Java 14, switch expressions were introduced exactly for this purpose.

Switch expression syntax 🙌

You can now do this using the old as well as new syntax:

```
//using the old syntax and the yield statement
double computeTax2(double income, int taxBracket){
    return income * switch(taxBracket){
        case 0 : yield 0.1; //observe the new keyword yield
        case 1 : { yield 0.2; } //enclosing the code within { } is optional
        default : yield 0.3;
    };
}

//or even better, using the new arrow → syntax
double computeTax3(double income, int taxBracket){
    return income * switch(taxBracket){
        case 0 → 0.1; //no yield statement required here
        //case 1 → { 0.2; } //can't use { } without a yield statement inside
        //case 1 → yield 0.2; //yield statement must be inside { }
        case 1 → { yield 0.2; } //fine
        default → 0.3;
    };
}
```

The above two code snippets illustrate almost everything there is to know about the switch expressions. Specifically:

1. You may write a switch expression using the old syntax as well as the new

1. syntax.
2. The value of a switch expression is generated by the expressions used alongside the switch labels either using a `yield` statement or by directly using an expression on the right side of the `→`.
3. The old syntax requires a `yield` statement, while the new syntax does not require it. If you want to use `yield` with the new syntax, you need to put the code within `{` and `}`.
4. You may use **any** expression in the switch blocks (and not just expression statements, as is the case with switch statements) as long as the type of the expression is compatible with the type of the value expected from the switch expression.

Now, coming to the subtle aspects of switch expression, which are not evident in the above code.

Value of a switch expression 🙋

Remember that an expression must have a value. Since a switch expression is an expression, it must also always have a value. This implies that the code inside a switch expression must generate a value for every possible value of the selector expression. Therefore, you cannot have a switch expression that has an execution path that doesn't generate any value. For example, the following expression is invalid.

```
return income * switch(taxBracket){  
    case 0 → 0.1;  
    case 1 → 0.2;  
};
```

What will happen if `taxBracket` is neither `0` nor `1` ? Well, there is no instruction about what to do in that case and thus, the given expression cannot be evaluated if the value of `taxBracket` is neither `1` nor `2` ! To avoid this issue, Java mandates that a switch expression must be "**exhaustive**", which means, it must provide an execution path that returns a value for every possible value of the selector expression. Thus, you

must either provide a case block for every value of the selector expression or provide a default block, which will ensure that a value is returned if no case label matches with the value returned by the selector expression.

Usually, when the selector is an enum, you are able to specify all the enum values explicitly as case labels, and in that case there is no need for the default path.

However, if a new enum constant is added later on to the existing enum, your old switch expression will fail to compile, and if you try to use the old precompiled class file, it will throw an exception at run time.

You may be tempted to provide a default path to make the switch expression "future proof", but that path will most likely be incorrect from the business logic perspective anyway. Since it is better to fail at compile time than at run time or proceed with incorrect results, in general, an exhaustive switch without a default clause is better than an exhaustive switch with one.

Type of the value of a switch expression 🙌

A switch expression is allowed to return different types of values from different paths of execution. For example:

```
int selector = //some value;
XXX value = switch(selector){
    case 0 → 0.1;
    case 1 → 2;
    default → 'x';
};
```

Observe that when selector is `0`, the switch returns `0.1`, which is a `double` value, when selector is `1`, the switch returns `2`, which is an `int` value, and when selector is neither `0` nor `1`, the switch returns `'x'`, which is a `char` value. So, now, the conundrum is how should the variable `value` be defined. If you define `value` as an `int`, obviously, a `double` or a `char` value cannot be assigned to it and the same argument holds if you define it as `char`. The only acceptable type is `double`, because a variable of type `double` can be assigned all three types of values.

Furthermore, since everything is an object in Java, you may also define `value` as `Object`. However, by defining `value` as `Object`, you lose all of the type information that you will have if you define it as a `double`. A `double` is more specific than `Object`.

For this reason, the type of switch expression is always determined by the compiler to be the most specific type that is compatible with all of the types of the values that the given switch expression can possibly return. It tries to keep as much of type information associated with the return value as possible. The following example illustrates the same point with objects instead of primitives:

```
XXX pet = switch(personType){  
    case 1 → new Cat();  
    default → new Dog();  
};
```

Assuming that `Cat` and `Dog` classes extend `Animal`, the type of the switch expression will be `Animal` because it is the most specific type that is compatible with `Cat` as well as `Dog`. Of course, since an `Animal` is also an `Object`, you may define the variable `pet` as `Object` as well. In strictly technical terms, the type of a switch expression is the most specific upper bound of all the possible types returned by the switch expression. Don't worry too much about it right now. It will be clear to you after you go through the chapter on Inheritance.

To apply the correct set of rules to a switch statement/expression, you need to first determine whether you are looking at a switch expression or a switch

statement. If you are in doubt, just see whether it is in a place where you would normally see an expression, such as on the right-hand side of an assignment operator, in a return statement, or as a method argument. If it is, then it is a switch expression, otherwise, it is a switch statement. Another quick test that you can do is, replace the whole switch code with a value such as a number or a string, and see if rest of the code still makes sense syntactically. If it does, it is a switch expression.

12.5 Pattern matching with switch (Java 21 Only)

We have already seen how an `if/else` statement can benefit from pattern matching with `instanceof`. Since a `switch` is conceptually similar to an if statement, it is natural to extend the feature of pattern matching to `switch`. The following code listing shows a simple decision making logic written using an `if/else` statement and a functionally equivalent `switch` statement written using new as well as old syntax:

```
Object obj = 10;

if(obj instanceof Integer i){
    System.out.println("Integer "+i);
}else if(obj instanceof Double d){
    System.out.println("Double "+d);
}else{
    System.out.println("Object "+obj);
}

//using the new arrow syntax
switch(obj){
    case Integer i → System.out.println("Integer "+i);
    case Double d → {
```

```

        System.out.println("Double "+d);
        break; //allowed but redundant
    }
    default → System.out.println("Object "+obj);
}

//using the old colon+break syntax
switch(obj){
    case Integer i : System.out.println("Integer "+i); break;
    case Double d : System.out.println("Double "+d); break;
    default : System.out.println("Object "+obj);
}

```

It will produce the following output:

```

Integer 10
Integer 10
Integer 10

```

The code looks quite straightforward. It tries to match the actual type of the object referred to by `obj` with the reference types given in the case labels. The block associated with the case label that matches with the type of the object is then executed. For each of the case labels, a pattern variable is also defined, which can be used within that case block. The switch statement with the old syntax works exactly the same way. Observe that it requires a `break` statement for each case block.

Let me now list the rules that are applicable **when using pattern matching with switch**:

1. **Exhaustive** - Recall that a regular switch statement need not be exhaustive but a switch statement that uses pattern matching is required to be exhaustive. A switch expression must always be exhaustive (irrespective of whether it is used with pattern matching or not).
2. **No fall through** - The "fall through" behavior that we saw with a regular case statement written using **old syntax** is not permitted when using pattern

2. matching. So, a `break` statement is **required** in each case block in the old syntax. The reason for prohibiting fall through is clear from the following code:

```
switch(obj){  
    case Integer i : System.out.println(i);  
    case String str : System.out.println(str.length());  
    default : System.out.println(obj);  
}
```

If `obj` points to an Integer and if the first case is allowed to fall through to the second one, there will be a problem because the second case block expects `str` to be initialized to point to a `String` ! However, it is possible to fall through to case null and default because no variables are expected to be available in these blocks. Therefore, the following is valid:

```
switch(obj){  
    case Integer i : System.out.println(i); break; //fall through is NOT allowed here  
    case String str : System.out.print(str.length()); //fall through is allowed here  
    case null : System.out.print("null"); //fall through is allowed here  
    default : System.out.print(obj);  
}
```

If `obj` points to `"hello"`, the above code will print `5nullhello` because of the fall through at `case String str:` and `case null :`.

3. **Pattern variable** - A pattern variable is required with each case label. The default label must not have a pattern variable. The scope of the pattern variable is the associated case block only. So, for example, the following is valid:

```
switch(obj){
```

```

3.  case Integer num : {
    System.out.println("Integer "+num);
    break; //break is required in old syntax
}
case Double num : { //ok to redefine num here
    System.out.println("Double "+num); break;
}
default : System.out.println("Object "+obj); //cannot use num
here
}

```

4. **Order of case labels** - The matching is done in the order of appearance of case labels. It is therefore, important to specify case labels in the order of increasing "dominance".

5. **Dominance of case labels** - A case label dominates another case label if every value that matches another case label also matches this case label but not vice-versa. For example, `case Object o` dominates `case Number n`, because every value that matches `Number` also matches `Object` (because every `Number` is an `Object`) but the reverse is not true (every `Object` is not a `Number`). Thus, you cannot put `case Object o` before `case Number n`. The reason is simple. If you do that, then `case Number n` will become unreachable. Allowing this may introduce hard to trace bugs in the code.

An interesting consequence of this requirement is that the `default` label can only be used at the end of the switch block because `default` dominates every other case label and any case label after default will be unreachable.

6. **Null** - `case null` is allowed. It does not dominate any other label and is dominated only by `default`. Thus, a `case null` may appear at any position but not after `default`.

It may seem unreasonable at first that a switch statement with pattern matching has to be exhaustive while the regular switch statement doesn't have to be. But pattern matching makes the switch statement amazingly powerful and complex. Unlike the old switch statement, which accepts only a limited type of values, a

switch statement with pattern matching has no limit on the type of objects it can work with. Therefore, it is easy to miss a case label inadvertently, which may introduce hard to trace bugs. Requiring a switch statement to be exhaustive ensures that an error will be generated by the compiler if the case labels do not cover all possible values and that will force you to pay close attention to the case labels. If you really want to omit some values, then you have to explicitly use the `default` label but it is advisable to avoid the default branch as much as possible. In fact, a non-exhaustive switch statement without pattern matching but written using the new syntax will also cause the compiler to issue a warning.

Only a "legacy switch statement", i.e., a switch statement without pattern matching written using the old syntax will compile without warning if it is not exhaustive going forward.

Enhancing case labels with guards

We discussed earlier that in a regular switch statement or expression, the case labels need to be compile time constant values. However, while using switch with pattern matching, we don't specify actual values. We only specify types. This means, after a required type is identified, we may still want to customize our logic based upon the value. For example:

```
Object obj = 10;
switch(obj){
    case Integer i → {
        if(i < 0) System.out.println("Negative value!");
        else System.out.println("Positive value!");
    }
    default → System.out.println("Object "+obj);
}
```

In the above code, we are customizing the logic based on the value of the `Integer`

object pointed to by `obj`. Since inspecting actual values is a common requirement while developing business logic, the case labels can now be "guarded" with the values that we are interested in, like so:

```
switch(obj){
    case Integer i
        when i < 0 → System.out.println("Negative value!");
    case Integer i
        when i ≥ 0 → System.out.println("Positive value!");

    default → System.out.println("Object "+obj);
}
```

Observe that the if/else statement has been replaced by two case-when blocks. The `boolean` expressions associated with the `when` clauses are called **guards**. Admittedly, the above example doesn't show a lot of benefit in terms of code length, but it does capture the business logic in a more intuitive fashion. You will find this feature more beneficial when the number of values you are interested in increases.

The **rule of dominance of case labels** takes a beating when using guards though. For example, in the following code you can see that the first case clearly dominates the second case, and so, the code should not compile. **But it compiles fine.**

```
Object obj = 1;
switch(obj){
    case Integer i
        when i > 0 → System.out.println("Positive value!");
    case Integer i
        when i == 1 → System.out.println("One"); //this is unreachable!
    default → System.out.println("Object "+obj);
}
//prints Positive value!
```

The reason the above code is acceptable to the compiler is that determining the

dominance of guarded case patterns is not possible for a compiler when the expressions used in the guard pattern can only be evaluated at runtime. Therefore, the compiler is **required to check** the dominance of such labels **only when** a case uses a guarded pattern with a constant expression. Something like this:

```
Object obj = "some string";
final int A = 10; //just trying to create a compile time constants here
final int B = 20;
switch (obj) {
    case String s when A<B → //this dominates the second case
        System.out.println("always true");
    case String s when "s".length() = 0 →
        System.out.println("got string S");
    default →
        System.out.println("Object " + obj);
}
```

The above code won't compile because the compiler can evaluate the constant expression used in the guard used in the first case and can figure out that the first case dominates the second case, which is unacceptable.

Using guard expressions that are not compile time constants can also make the compiler deem a switch to be not exhaustive and reject the code. For example, assuming that `obj` is an `Integer` variable, `case Integer i when i<0` and `case Integer i when i≥0` seem collectively exhaustive to humans but not to the compiler because `i<0` and `i≥0` are not compile time constant expressions.

Using Switch with records

Since a record class is just like any other regular class, the above rules apply to records as well. But as you saw earlier in the **Operators** chapter, where we deconstructed records with `instanceof` using pattern matching, you can do the same thing in a

`switch` as well. Here is the same example using a `switch` statement:

```
public record Student(int id, String name){ }
void main(){
    Object obj = new Student(1, "Bob"); //constructing a Student
    object
    switch(obj){
        case Student(int i, String n) → //deconstructing the Student
        object
            System.out.println("Id = "+i+" Name = "+n);
        default → System.out.println(obj);
    }
}
```

The above code is a simple `switch` statement written using the new arrow syntax. In the first case label, besides just matching the variable `obj` with the `Student` record, we are also extracting the values of `Student`'s fields into variables `i` and `n`.

Just like with `instanceof`, you may do the deconstruction of nested records in a switch as well. For example:

```
record Address(String street){ }
record Student(int id, String name, Address addr){ }
void main(){
    Object o = new Student(1, "Bob", new Address("123 Main"));
    switch(o){
        case Student(int i, String n, Address(String value)) →
            System.out.println(value); //prints 123 Main
        default → System.out.println(" got o");
    } }
}
```

Observe that the names of the variables used during deconstruction need not match the names of the component fields of the record.

To sum up all of the enhancements to the switch statement/expression in Java 21, a switch block may now have:

- **case constants** - Examples: `case 1`, `case "yes"`, `case MONDAY`
- **case null** - Requires all switch statements and expressions to be exhaustive. `null` does not dominate anything. Nothing dominates `null` except `default`.
- **case patterns** - Examples: `case String s`, `case CheckingAccount ca`
- **case patterns with guards** - Examples: `case String s when s.startsWith("-")`, `case Integer i when i==0`
- **case patterns with record deconstruction** - Example: `case Student(int id, String name)`
- **case patterns with record deconstruction and guards** - Example: `case Student(int i, String n, Address(String value)) when value.startsWith("123")`
- **default** - All kinds of switch statements and expressions can have the default branch. But in case of a switch with pattern matching, the default branch is allowed only if the switch is not already exhaustive. For example:

```
switch(o){ case Integer i → print("Int "+i); case Object o → print("obj "+o); default → print("def"); } //case Object o covers all possibilities making default unreachable
```

Pattern matching with records and sealed classes adds substantial complexity in applying the switch rules related to its exhaustiveness and dominance of case labels. One has to be a human Java compiler to determine if a given switch code is really exhaustive if it involves classes, interfaces, and nested records. This task however can be easily done by IDEs. Fortunately, the exam does not ask questions based on contrived code snippets. If you just understand the basic principles explained in this section and do a few mock questions, you will be well prepared for the exam.

12.6 Switch statement and expression Summary 📌

Switch Statement (without pattern matching) Rules Summary

	Old Syntax (:)	New Syntax(->)
	Allows zero or more individual statements or zero or more code blocks.	Allows either exactly one statement or exactly one code block.
	Examples (all valid):	Examples (all invalid):
	<pre>case 0: case 1: stmt1; stmt2; case 2: { <code block1>} { <code block 2> }</pre>	<pre>case 0 → //invalid, empty case case 2 → stmt1; stmt2; //invalid, more than one statement case 4 → { } { } //invalid, more than one block</pre>
Selector expression type	Limited to: byte, short, int, char, and their wrappers, String, and enum. Not Allowed: boolean, long, float, and double .	Limited to: byte, short, int, char, and their wrappers, String, and enum Not Allowed: boolean, long, float, and double
Case label constant type	Must be compatible with selector expression type	Must be compatible with selector expression type Allows multiple case constants separated by comma. Example:
case label	Allows a single case constant.	<pre>case 1, 2 → { }</pre>

default label	Not required	Not required
case null	Allowed in Java 21 only if the switch statement is exhaustive and follows the rule for dominance of case expressions.	Allowed in Java 21 only if the switch statement is exhaustive and follows the rule for dominance of case expressions.
Order of switch labels	No restriction	No restriction
break	Needed only to prevent fall through	Prohibited when a switch label is associated with a single statement.
yield	Not allowed	Allowed within the code block associated with a switch label but is redundant.
exhaustive	Not necessary	Not allowed
		Not necessary

Switch Expression (without pattern matching) Rules Summary

Old Syntax (:) New Syntax(->)

Allows one expression or one code block. No `yield` required for single expression but required only within code block.

Examples:

```
case 1 → 1; //valid, returns 1, no yield required
case 3 → "a".length(); //valid, returns 1, no yield required
case 4 → { yield 1; } //valid, yield is required in a block
```

case 0 → //invalid, missing expression

case 2 → yield 1; //invalid, must not write yield

Allows one expression or one code block. Requires `yield` to cause it to return value.

Examples:

```
case 0: yield 1; //valid, returns 1
case 3: yield "aa".length(); //valid, returns 2.
case 2: "a".length(); //syntactically valid, no yield required for non-void method call but falls through without returning a value
```

case 1: 1; //invalid, missing yield keyword

Limited to: `byte`, `short`, `int`, `char` and their wrappers, `String`, and `enum` **Limited to:** `byte`, `short`, `int`, `char` and their wrappers, `String`, and `enum`

Not Allowed: `boolean`, `float`, `long`, and `double` **Not Allowed:** `boolean`, `float`, `long`, and `double`

Selector expression type

Case label constant type

Must be compatible with selector expression type. Must be compatible with selector expression type.

case label

Allows a single case constant.

Allows multiple case constants separated by comma.

Example: `case 1, 2 → 1;`

default label	Not required if already exhaustive	Not required if already exhaustive
case null	Allowed in Java 21 but must follow the rule for dominance of case expressions.	Allowed in Java 21 but must follow the rule for dominance of case expressions.
Order of switch labels	No restriction	No restriction
break	Prohibited	Prohibited
yield	Required to return value	Required only when returning a value from a code block. Otherwise prohibited.
exhaustive	Necessary	Necessary

Switch Statements and Expressions (with pattern matching) Rules Summary (Java 21 Only)

	Old Syntax (:)	New Syntax(->)
Selector expression type	All reference types are allowed	All reference types are allowed
Type of case constants	Must be compatible with the type of the selector expression	Must be compatible with the type of the selector expression
exhaustive	Necessary	Necessary
default label	Not required if already exhaustive	Not required if already exhaustive
case null	Allowed May appear anywhere except after <code>default</code> label.	Allowed May appear anywhere except after <code>default</code> label.

Order of switch labels	Must follow the rule for dominance of case expressions.	Must follow the rule for dominance of case expressions.
	<code>default</code> must be the last label, if present.	<code>default</code> must be the last label, if present.
Fall through behavior	Not allowed. So, <code>break</code> is required after each switch label except when falling through to <code>case null</code> and <code>default</code> .	Fall through is not possible. <code>break</code> is prohibited.

Tips to work out code containing switch

The new enhancements to `switch` have truly turned it into a monster. It is extremely powerful but its syntax has so many ifs and buts that it is not possible for normal humans to remember and apply all of the rules and figure out how a switch will behave during an exam!

My goal here is not to reproduce all the rules of switch, which you can find by going through JEP 441 if you like, but to show you only those rules that are essential for correct development and for answering questions in the exam.

Based on experience, I can tell you that exam does not try to trick you with weird switch code but expects you to know and apply the following points to arrive at the answer:

1. Check whether the switch uses pattern matching or not by looking at two things. First, if the type of the switch selector expression is anything other than the legacy types, i.e., `String`, `byte/Byte`, `char/Character`, `short/Short`, `int/Integer`, or enums, it is a switch with pattern matching. Second, if the case labels use pattern matching expressions such as `case String s`, it is a switch with pattern matching. A switch with pattern matching **must be exhaustive, must follow the rule on dominance of case labels, and must not exhibit fall through.**

2. Determine whether the given switch is a statement or an expression. If it is a switch **expression**, it **must be exhaustive**.
3. Check whether the switch uses the old or the new syntax. Old syntax falls through by default and needs `break` and `yield` to prevent fall through. New syntax doesn't fall through. Check if the behavior of the given switch is compatible with the above two points.

Here are a few examples, that show how to apply the above rules.

Example 1:

```
Integer obj = 1 ;
switch(obj){
    default : System.out.println("Object "+obj);
    case 1 : System.out.println("Integer 1");
    case 2 : System.out.println("Integer 2");
}
```

The above code is a regular switch statement and it uses the old syntax. It does not need to follow the dominance rule and it may exhibit fall through. So, it is fine. It prints:

```
Integer 1
Integer 2
```

Example 2:

```
String str = "1";
switch(str){
    case "" : System.out.println("String empty");
    case "1" : System.out.println("String 1");
    case String s : System.out.println("String");
}
```

```
case String s when s.startsWith("a") : System.out.println("String a");
default : System.out.println("Object "+obj);
}
```

Although this switch uses a legacy type as the selector expression, its case labels use pattern matching. Therefore, it is a switch with pattern matching. Applying point 1, we can see that it is exhaustive but it does not follow the dominance rule because `String s` dominates `String s when s.startsWith("a")` but appears earlier in the case labels. Furthermore, all values of `str` will satisfy `case String s` as well as the `default` case. Therefore, both cases dominate each other and so, `case String s` and `default` cannot coexist. Also notice that it uses old syntax, which falls through by default. This too is unacceptable for a switch that uses pattern matching.

It may be fixed like this:

```
String str = "1";
switch(str){
    case "" : System.out.println("String empty"); break;
    case "1" : System.out.println("String 1"); break;
    case String s when s.startsWith("a") : System.out.println("String a");
break;
    case String s : System.out.println("String "+str);
}
```

12.7 Quiz 🙌

Q1. Identify all issues with the following code:

```
void main(){
    Object option = "donothing";
}
```

```

switch(option){
    case "dothis" : dothis(); break;
    case "dothat" → dothat();
    case "donothing":
    case null :
}
}
static void dothis(){ }
static void dothat(){ }

```

Answer:

Let's start from the top.

1. The type of the `option` variable is `Object`. The regular switch (i.e. switch without pattern matching) does not accept `Object` as a selector expression type. It accepts only a limited set of types, which includes `String`. So, you have to either convert the case labels into case labels with pattern matching (with or without guards, such as `case String s:` or change the type to `String`. Let us change it to `String`.
2. You cannot mix the old and the new syntax in the same switch. Therefore, let's change `→` to `:`. (As an exercise, change `:` to `→` and then find out the issues.)
3. The usage of `case null` implies that you are using the new features of switch and that requires a switch to be **exhaustive**. So, either remove this case or add the `default` label. Either option will do.

Q2. Identify all issues with the following code:

```

void main(){
    String value = " 123 ";
    Object option = "trim";
    int len = switch(option){

```

```
    case String s : value.length();
    case String s when s.equals("trim") : value.trim().length(); break;
    default : 0;
};
System.out.println("len is "+len);}
```

Answer:

Let's start from the top again.

1. The switch appears on the right-hand side of the assignment operator. This means, it is a switch expression. That's fine but we have to make sure that the switch is exhaustive. Since there is a default branch, it is indeed exhaustive. Good.
2. The type of the `option` variable is `Object` and our case labels use pattern expressions, so, this is good too.
3. It is only using the old syntax for case labels. That's fine. But with old syntax, you must use the `yield` keyword to return the value. Furthermore, you cannot use `break` because a switch expression does not have fall through behavior.
4. While using pattern matching, dominance of case labels must also be checked. Here, `case String s` dominates the second case and so, it must appear later.

Q3. Identify all issues with the following code:

```
String winLoseOrTie(Object obj){
    return switch(obj){
        case null → "tie";
        case Integer i
            when i<10 → "lose";
        case Integer i
            when i>10 → "win";
        case Integer i
            when i=10 → "tie";
```

```
};  
}
```

Answer:

Since this switch code returns a value, it must be a switch expression, which means, it must be exhaustive. The type of the switch condition variable is `Object` but the cases in the switch cover only `Integer` values. What if `obj` points to some other type of object such as a `String`? The given switch expression will not be able to return any value in that case, which means it is not exhaustive. default -> "tie"; case should be added at the bottom to fix it.

Q4. What if the type of the method parameter is changed to `Integer` instead of `Object` in the above code? Will that make it exhaustive?

Answer: No, because the compiler still cannot confirm that it covers all possible values that `obj` may refer to. Remember that the compiler cannot evaluate expressions such as `i < 10` because they are not compile time constant expressions.

12.8 Exercise

1. Write a method that accepts a number as input and prints whether the number is odd or even using an if/else statement as well as a ternary expression.
2. Accept a number between 0 to 5 as input and print the sum of numbers from 1 to the input number using code that exploits the "fall through" behavior of a switch statement.
3. Accept a number as input and generate output as follows using a cascaded and/or nested if/else statement - if the number is even print "even", if it is divisible by 3, print "three", if it is divisible by 5, print "five" and if it is not divisible by 2, 3, or 5, print "unknown". If the number is divisible by 2 as well as by 3, print "23", and if the number is divisible by 2, 3, and 5, print "235".

4. Indent the following code manually such that it reflects correct if - else associations. Use a plain text editor such as Notepad. Copy the code into a Java editor such Netbeans or Eclipse and format it using the editor's auto code formatting function. Compare your formatting with the editor's.

```
int a = 0, b = 0, c = 0, d = 0;
boolean flag = false;
if (a == b)
if (c == 10)
{
    if (d > a)
    {
    } else {
    }
    if (flag)
        System.out.println("");
    else
        System.out.println("");
}
else if (flag == false)
    System.out.println("");
else if (a + b < d) {
    System.out.println("");
}
else
    System.out.println("");
else d = b;
```

5. Rewrite the following code in four ways - using a switch statement and a switch expression, with as well as without using pattern matching.

```
Object obj = null;
int iVal;
if (obj instanceof String) {
```

5.

```
String str = (String) obj;  
if (str.startsWith("a")) {  
    iVal = 96;  
} else if (str.startsWith("A")) {  
    iVal = 65;  
} else iVal = 0;  
} else iVal = 0;
```